

# SIH 2026 — College Portal · Architecture Document

**Project:** Smart India Hackathon 2026 — institutional promotion, awareness, internal registration, two-level screening, and results portal. **Stack:** React + TypeScript (Vite) · FastAPI (Python) · MongoDB (Atlas) · Cloudinary object storage · JWT auth with role-based access control. **Version:** 1.0 · **Owner:** student dev team · **Built with** Claude Code.

---

## 1. Purpose and scope

This portal serves three audiences from one codebase.

Visitors arrive to learn what SIH is, why the college is running an internal hackathon, and how to take part; they read awareness content and register a team. Participants, once registered, track their progress through two screening rounds from a private dashboard and submit their work at each level. Coordinators and the college SPOC review those submissions, score them, publish announcements, and finally publish the list of teams selected for the national SIH.

The design goal is a portal that looks and behaves like a real institutional product — coherent design language, a documented API, a clean permission model, and an auditable screening process — rather than a set of loosely assembled pages.

The named people configured into the content layer are the college SPOC, Bhargav R, and the student coordinators, Partha Shankar and Nirmith M Jain. Their profiles and photos are stored as content records (see section 5), not hard-coded, so they can be edited from the admin panel.

---

## 2. Technology choices and rationale

The frontend is React with TypeScript, built with Vite. TypeScript is used throughout because static types catch integration mistakes before runtime and because it is the strongest single signal of production-quality frontend work. Styling uses Tailwind CSS with a small custom design-token layer so the visual system stays consistent. Client-side routing uses React Router, server state is managed with TanStack Query (React Query) so that loading, caching, and refetching are handled uniformly, and scroll and reveal animations use Framer Motion.

The backend is FastAPI. It was chosen over Flask for three concrete reasons: request and response bodies are validated automatically by Pydantic models, the framework is async-first, and it generates interactive OpenAPI (Swagger) documentation at `/docs` with no extra work. That auto-generated API documentation doubles as a demo asset when presenting the project.

The database is MongoDB, hosted on MongoDB Atlas (free tier is sufficient). The data model is designed around a single aggregate document per team — members and both screening rounds are embedded on the team record — so the participant dashboard and the admin review console each load from one read. Cross-document references are kept to the

few places they genuinely belong (a team references its problem statement and its leader account).

Uploaded binaries — profile photos, presentation decks, prototype files — are never stored in the database. They go to Cloudinary (or S3) and only the resulting URL is stored on the relevant document. This keeps documents small and reads fast.

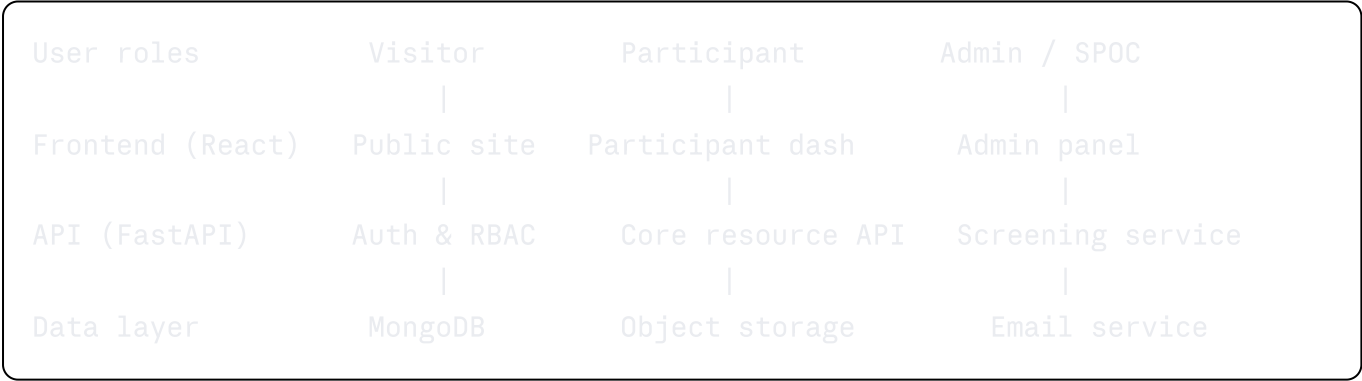
Authentication is stateless JWT with role-based access control enforced on every protected endpoint.

### 3. System architecture

The system is a single-page React application talking to a FastAPI backend over a JSON REST API, with MongoDB, object storage, and an email service behind the API.

There are four logical layers. At the top are the four user roles. Below them are the three frontend surfaces they use: the public site, the participant dashboard, and the admin panel — all served from one React build, with routing deciding what each role can see. Below the frontend is the API layer, which splits into authentication and role guards, the core resource endpoints, and the screening service that owns the two-level review workflow. At the bottom is the data layer: MongoDB for structured data, object storage for files, and an email service for status notifications.

A request flows top to bottom. The browser calls the API with a bearer token; the auth layer verifies the token and the caller's role; the endpoint runs, reading or writing MongoDB and, where relevant, signing an upload URL or queuing an email; the response returns as JSON that React Query caches on the client.



### 4. Roles and access control

There are four roles, arranged so that each higher role can do everything the one below it can.

A participant can register a team, edit their own team while registration is open, submit work at each screening level, and read their own team's status and any published announcements. They can never see other teams' data or any review scores except the feedback left on their own team.

A coordinator (Partha Shankar, Nirmith M Jain) can list and open all teams, review and score Level 1 and Level 2 submissions, leave feedback, and post announcements.

The SPOC (Bhargav R) can do everything a coordinator can, and additionally publish the final results and perform overriding approvals.

An admin can do everything above, plus manage user accounts and edit the content pages (principal's message, coordinator profiles, awareness copy).

This is implemented as JWT-based authentication with role checks on the server. Every protected route declares the minimum role it requires, and a FastAPI dependency rejects any request whose token does not meet it. The client also hides UI that the user's role cannot use, but the server is the single source of truth — the client-side hiding is only for user experience, never for security.

---

## 5. Data model (MongoDB collections)

The model favours embedding data that is always read together and bounded in size, and referencing data that is shared or unbounded.

### users

One document per account.

```
{
  _id, name, email (unique, indexed),
  passwordHash,           // argon2 or bcrypt – never plaintext
  role,                   // "participant" | "coordinator" | "spoc" |
  "admin"
  photoUrl,               // Cloudinary URL, nullable
  department, year,
  createdAt
}
```

### teams

The aggregate root. Members and both screening rounds are embedded so a dashboard or review screen loads in one read.

```
{
  _id, name,
  leaderId,           // ref -> users._id
  problemStatementId, // ref -> problemStatements._id, nullable until
chosen
  theme,
  members: [          // embedded, bounded (SIH cap is 6)
    { name, email, department, year, role }
  ],
  status,             // overall lifecycle state, see section 6
  screening: {
    level1: { status, score, feedback, submissionUrl, reviewerId, reviewedAt
  },
    level2: { status, score, feedback, submissionUrl, reviewerId, reviewedAt
  }
  },
  createdAt, updatedAt
}
```

Indexes: `leaderId`, `status`.

#### `problemStatements`

The SIH problem statements, browsable by everyone.

```
{ _id, sihId, title, organization, theme, category, description, difficulty }
```

Indexes: `theme`, and a text index on `title` + `description` for the explorer's search box.

#### `announcements`

```
{ _id, title, body, category, authorId, publishedAt, isPublished }
```

#### `content`

A small CMS so institutional copy is editable from the admin panel rather than living in code. Keyed by slug.

```
{ _id, slug, type, payload }
// slugs: "principal-message", "profile-bhargav", "profile-partha",
//        "profile-nirmith", "awareness-why-sih", "awareness-innovation"
```

Profile records carry name, role, photoUrl, bio, and contact/LinkedIn in `payload`.

## auditLog

Every screening transition is recorded here, giving the review process a tamper-evident trail.

```
{ _id, teamId, action, fromStatus, toStatus, actorId, timestamp }
```

## 6. Screening workflow (state machine)

Screening is modelled as an explicit state machine rather than a scatter of boolean flags. A team's `status` field holds exactly one state, and only defined transitions are allowed; the screening service is the only code permitted to move a team between states, and every move writes an `auditLog` entry.

```
registered
-> l1_submitted      (participant uploads Level 1 work)
-> l1_under_review   (a coordinator opens it)
-> l1_cleared        (score recorded, decision = pass)    or l1_rejected
-> l2_submitted      (participant uploads Level 2 work)
-> l2_under_review
-> selected          (SPOC publishes)                    or l2_rejected
```

Participants can only trigger the two submission transitions, and only when the team is in the correct preceding state. Reviewers trigger the review and decision transitions. Publishing to `selected` is reserved for the SPOC. A rejected team is terminal unless an admin explicitly reopens it.

Because the current state and the last two decisions live on the team document, the participant dashboard renders the whole journey from a single read, and the admin funnel view aggregates over the `status` field.

## 7. API design

The API is REST over JSON, versioned under `/api/v1`, and fully described by the auto-generated OpenAPI page at `/docs`. Requests and responses are validated by Pydantic models. Each endpoint below notes the minimum role required.

Authentication: `POST /auth/register` and `POST /auth/login` are public; `GET /auth/me` returns the current user.

Teams: `POST /teams` creates a team (participant); `GET /teams/mine` returns the caller's team (participant); `GET /teams` lists and filters all teams (coordinator+); `GET /teams/{id}` and `PATCH /teams/{id}` read and edit a team.

Submissions and uploads: `POST /uploads/sign` returns a signed upload target for object storage (any authenticated user); `POST /teams/{id}/submissions` records a submission URL against a level (participant, gated on state).

Screening: `(POST /teams/{id}/screening/{level}/review)` records a score, feedback, and a pass/fail decision, and drives the state machine (coordinator+).

Problem statements: `(GET /problem-statements)` searches and filters (public); create, update, and delete are admin-only.

Announcements: `(GET /announcements)` is public; create, update, and delete are coordinator+.

Content: `(GET /content/{slug})` is public; `(PUT /content/{slug})` is admin-only.

Results: `(GET /results)` returns the published selected teams (public); `(POST /results/publish)` is SPOC-only.

Statistics: `(GET /stats/public)` returns the counts the landing page animates (teams registered, ideas submitted, problem statements available, days to deadline); `(GET /stats/admin)` returns the screening funnel (coordinator+).

---

## 8. Frontend structure

The frontend is one React build whose routing separates the three surfaces. Public routes render without a token. Dashboard and admin routes are wrapped in a `(ProtectedRoute)` component that checks the decoded token's role and redirects otherwise.

```
frontend/src/
  api/           axios client + typed TanStack Query hooks
  components/    reusable UI primitives and layout shells
  features/      auth, teams, screening, announcements (logic + views)
  pages/
    public/      landing, awareness, timeline, principal, team, register,
                  problem-statements, results, faq, contact
    dashboard/   team profile, status, submissions, announcements
    admin/       registrations, screening console, announcements editor,
                  results publisher, content editor, analytics
  routes/        route table + ProtectedRoute
  lib/           design tokens, formatting, auth helpers
  styles/        Tailwind config + token layer
```

Server state (teams, announcements, stats) is owned by TanStack Query so caching and refetching are consistent; only genuinely local UI state uses React's own state hooks. The design tokens in `(lib)` are the single place colours, spacing, and typography are defined, which is what keeps the site from looking assembled.

---

## 9. Backend structure

```
backend/app/  
  main.py          app factory, CORS, router registration  
  core/            config (env), security (JWT, hashing), dependencies (role  
guards)  
  db/              Mongo client, index creation on startup  
  models/          Pydantic request/response + document schemas  
  routers/         auth, teams, submissions, screening, problem_statements,  
announcements, content, results, stats  
  services/        screening state machine, email, storage (Cloudinary  
signing)  
  tests/           pytest  
Dockerfile  
pyproject.toml
```

Business rules that are more than a single query — chiefly the screening state machine and the email notifications that fire on a status change — live in `services`, so routers stay thin and the rules are unit-testable in isolation.

---

## 10. Security and non-functional requirements

Passwords are hashed with argon2 (or bcrypt) and never stored or logged in plaintext. JWTs are short-lived access tokens; treat a longer-lived refresh token as an optional later addition. All configuration and secrets — database URI, JWT signing key, Cloudinary credentials, SMTP details — come from environment variables and are never committed. CORS is restricted to the deployed frontend origin. Every input is validated by Pydantic before it reaches business logic, and the authentication endpoints are rate-limited to blunt brute-force attempts. All traffic is HTTPS in production, which the hosting platforms provide by default. File uploads are constrained by type and size at the point the upload URL is signed.

---

## 11. DevOps and deployment

The repository is a monorepo with `frontend/` and `backend/` beside a root `docker-compose.yml` for local development (it brings up the API and a local MongoDB together), this document, and a README. A GitHub Actions workflow runs linting and tests on every push, giving the project a visible CI badge.

In production the frontend deploys to Vercel, the FastAPI backend to Render or Railway, the database to MongoDB Atlas, and files to Cloudinary — all of which have free tiers adequate for this project. Environment variables are set in each platform's dashboard rather than in code.

---

## 12. Creative and design direction

The site is built around one committed idea: the Innovation Pipeline. Every idea's journey — from a raw concept, through Level 1 and Level 2 screening, to the SIH grand finale and real-world impact — is the organising metaphor. It appears as the hero motion on the landing page, as the "how it works" explainer, and, most usefully, as the live screening funnel on the results and admin views. Reusing one metaphor across three places is what makes the site read as designed rather than assembled.

The palette is a restrained tricolor for light mode: deep saffron and forest green used only as accents over a cream and white canvas, with an ink-navy for body text — a nod to the national character of SIH without imitating a flag. A distinctive display typeface is paired with a clean sans for body copy. The recurring background motif fuses a geometric rangoli or Ashoka-chakra pattern with faint circuit-board tracery, giving the tradition-meets-technology theme a visual signature.

Four features carry most of the impact. The landing page animates real counters pulled from `/stats/public`. The problem-statement explorer is a searchable, filterable card grid over the SIH themes. The awareness section — why SIH matters and why innovation and education matter — is delivered as a scroll-driven data story rather than a wall of text. The principal's message is treated as a designed centrepiece with portrait or short video. A certificate generated for each selected team is a small, demo-friendly finishing touch.

---

## 13. Build roadmap

Milestone one establishes the foundation: scaffold the monorepo, wire up authentication and the role guards, and confirm the four roles behave correctly end to end.

Milestone two builds the public site and the content layer — landing, awareness, timeline, principal's message, and the coordinator profiles for Bhargav, Partha, and Nirmith editable through the content collection.

Milestone three delivers registration and team creation, then the participant dashboard with the two submission steps and object-storage uploads.

Milestone four builds the admin panel: the registrations table, the screening console that drives the state machine, and the funnel analytics.

Milestone five adds announcements, results publishing, and certificate generation.

Milestone six is polish and release: the full design system and animations, Dockerisation, the CI workflow, and deployment to Vercel, Render, and Atlas.

Working in this order means there is a demonstrable, deployable slice of the product at the end of every milestone.